Advanced Compilers

Project 3

Virtual Machine & Optimization Laboratory

Dept. of Electrical and Computer Engineering

Seoul National University

Index

1. Sparse Conditional Constant Propagation (SCCP)
2. Project 3
3. Code Review

SCCP

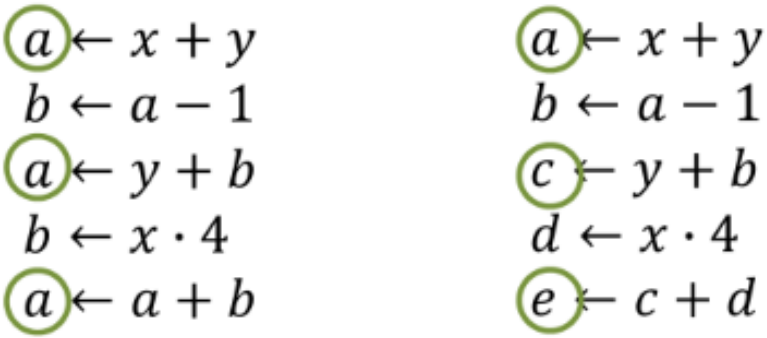
Static Single Assignment (SSA) Form

Standard representation in most compilers

Normal form ➔ SSA form ➔ Analysis and optimization ➔ Normal form

What is SSA form? "Single" assignment

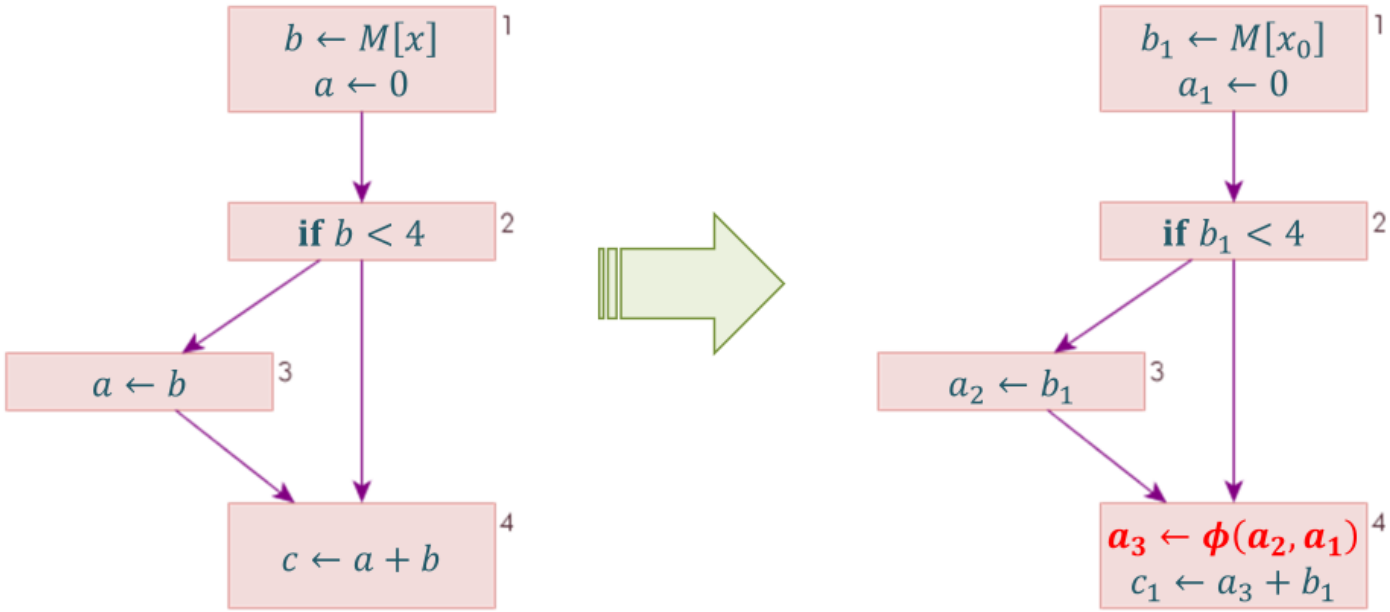
- Only one definition site for each variable: similar to live range renaming



At Control Join Point: Add ϕ-function

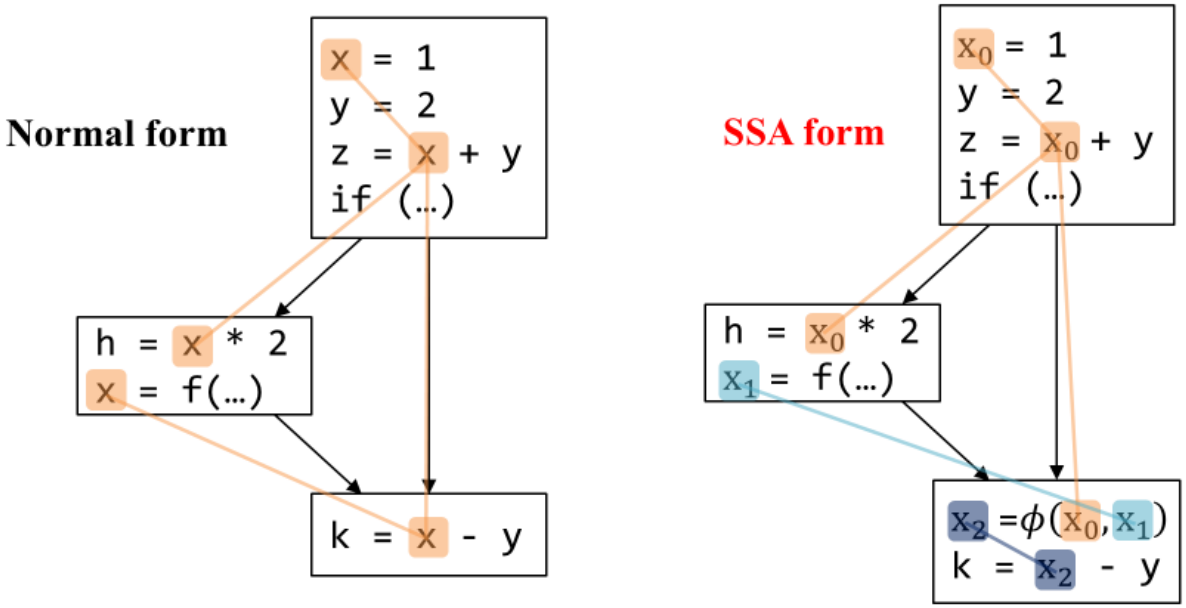
A notational fiction

- Not an ordinary function
- Return value depends on control path
- Not actually executed, but used for analysis and optimization



Why SSA? Simpler Analysis and Optimization

e.g., constant propagation is easy; for any x=c, replace use of x by c

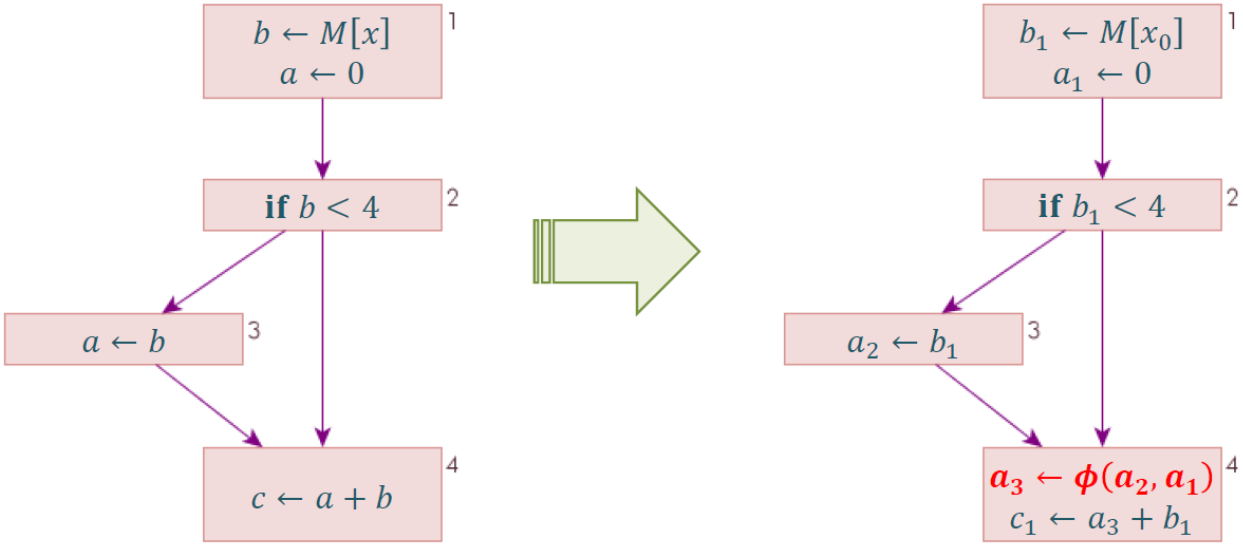


- Normal form
  - ✓ It is not clear which x to be propagated by x = 1
- SSA form
  - ✓ We can decide it simply based on the variable name

Recap: SSA

Static Single Assignment (SSA)

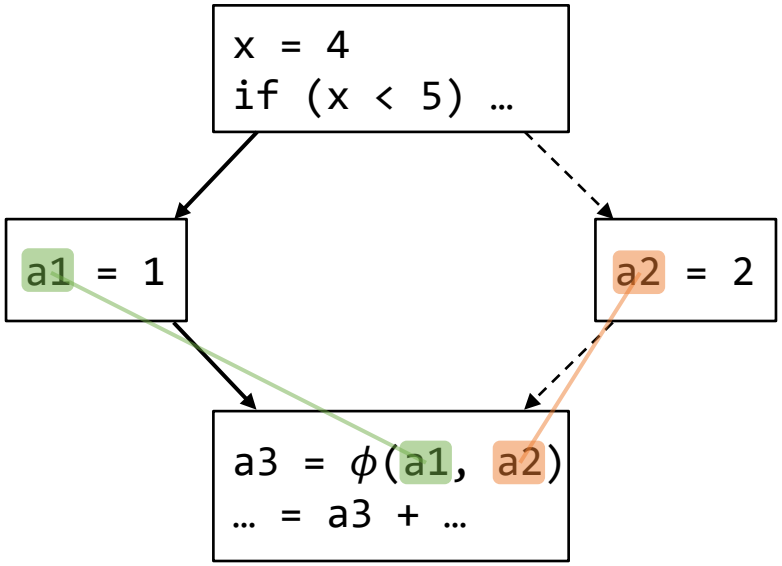
- 각 변수는 한 번만 정의됨 (single assignment)
- 각 use는 unique한 definition에만 연결됨
- Control-flow의 merge는 phi라는 가상의 Instruction으로 명시적으로 표현됨



SCCP

Sparse Conditional Constant Propagation (1991)

- Conditional Constant
  - 계산 가능한 조건 (conditional, predicate) 들을 반영한, control-flow를 분석
    ⇒ 더 세밀한 분석
  - **e.g.**,



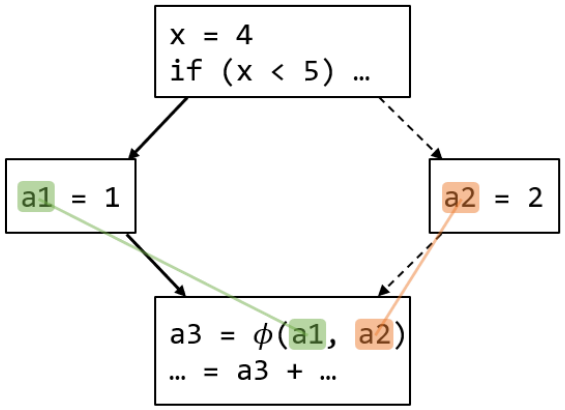
  - 동적 실행 흐름 (dynamic control-flow) 을 고려하면 a3도 constant

SCCP

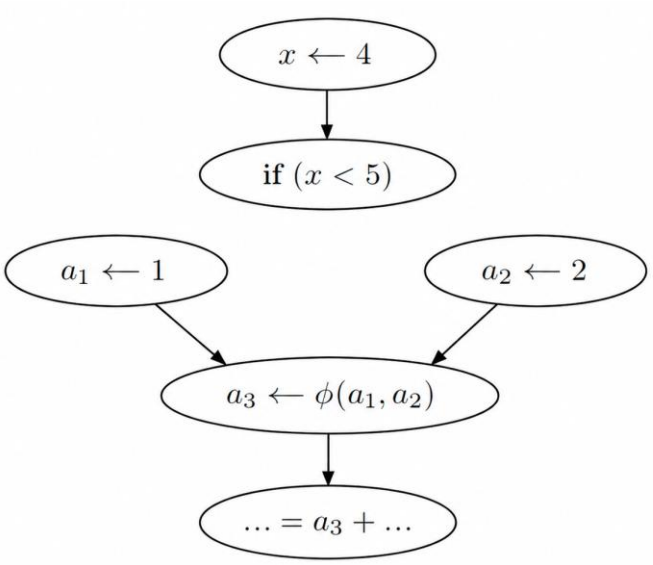
Sparse Conditional Constant Propagation (1991)

- Sparse
  - Dataflow information을 CFG 뿐만 아니라 SSA graph를 통해서 전파
    ⇒ 더 빠른 수렴

  - SSA Graph
    - *Use-Def* chain의 묶음



CFG



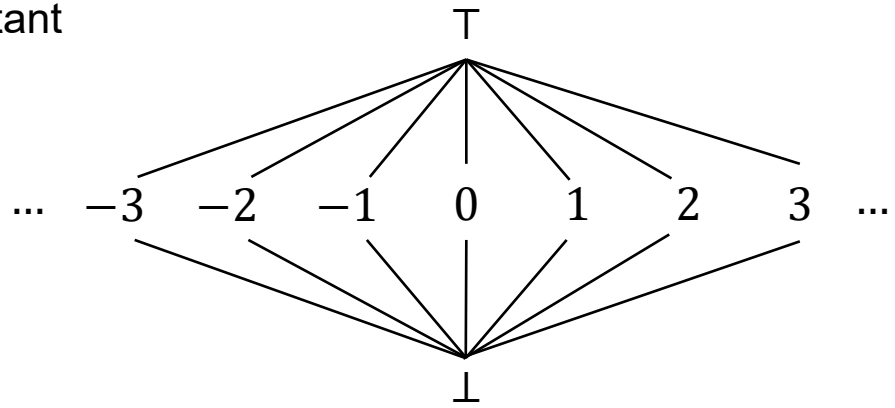
SSA Graph

  - SSA form의 dominating relationship 덕분

SCCP

Sparse Conditional Constant Propagation

- Lattice $(V,\sqcap)$
  - 이번 과제에서는 정수 값, 일부 단순한 연산들에 대해서만 고려
  - $\top$: Undefined
  - $\bot$: Not a constant



- Meet

$$
\begin{aligned}
x \sqcap \top &= x \\
x \sqcap \bot &= \bot \\
c_1 \sqcap c_2 &= c_1 \quad (c_1 = c_2) \\
c_1 \sqcap c_2 &= \bot \quad (c_1 \neq c_2)
\end{aligned}
$$

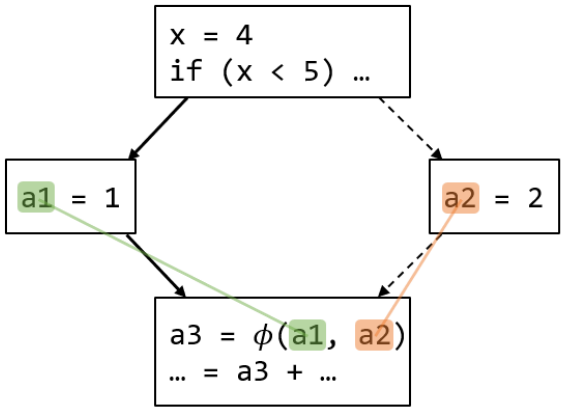
SCCP

Sparse Conditional Constant Propagation

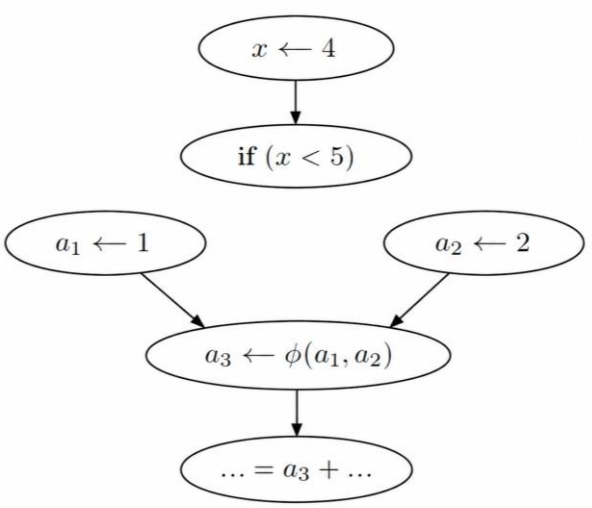
- Algorithm
  - SCCP Main Loop

Algorithm 1 Sparse Conditional Constant Propagation

```
Input: CFG, SSA graph
CFGWorklist ← {Edge(NULL, entry)}
SSAWorklist ← ∅
while (!CFGWorklist.Empty || !SSAWorklist.Empty) do
    x ←pop one from any list
    if x is from CFGWorklist then
        E ← x
        B ←destination(E)
        markExecutable(E)
        visit(φ)    (∀φ ∈ B)
        if B.IsFirstVisit then
            visit(I)    (∀I ∈ B)
        if hasSingleSuccessor(B) then
            S ←getSingleSuccessor(B)
            E' ←Edge(B, S)
            if !isExecutableEdge(E') then
                CFGWorklist.append(E')
    else /* x is from SSAWorklist */
        if x is a φ then
            visit(x)
        else
            if Any incoming CFG edge to x is executable then
                visit(x)
```



CFG



SSA Graph

SCCP

Sparse Conditional Constant Propagation

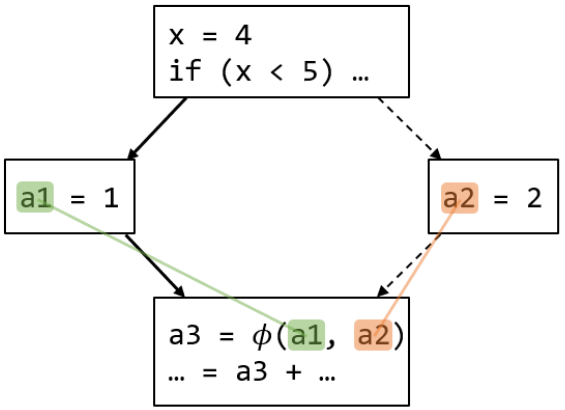
- Algorithm
  - Visit Function

Algorithm 2 visit

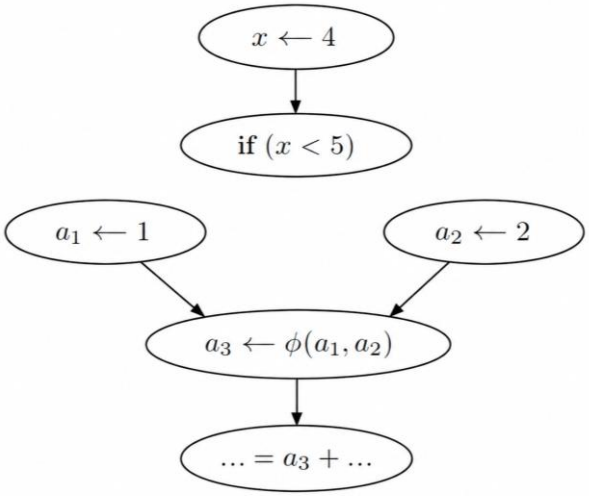
Input: An instruction I

if I is a ϕ **then**
  Meet all the arguments from the executable edges.
else if I is a conditional `branch` **then**
  Evaluate the condition and append reachable edge to `CFGWorklist`.
else
  Update dataflow information using transfer function.

if Dataflow information of I had been changed **then**
  Append all outgoing SSA edges of I to `SSAWorklist`.



CFG



SSA Graph

SCCP - How it works

Sparse Conditional Constant Propagation

- CFG Worklist + SSA Worklist
  - 크게 control-flow edge로, 세세하게 SSA edge로 dataflow analysis 정보 전달

1. Initialize CFGWorklist, SSAWorklist
2. **for** **NOT** CFGWorklist.empty || **NOT** SSAWorklist.empty**:**
   1. $x \leftarrow$ pop(CFGWorklist + SSAWorklist)

- ϕ와 다른 operation들을 구분해서 방문 (`visit`)
  - ϕ는 항상 최신의 정보를 반영할 필요가 있음

- LLVM의 구현
  - https://github.com/llvm/llvm-project/blob/llvmorg-19.1.1/llvm/lib/Transforms/Utils/SCCPSolver.cpp#L1897

SCCP - How it works

Sparse Conditional Constant Propagation

- CFG Worklist (x is an edge (block) from CFGWorklist)
  - $B \leftarrow x.\text{Destination}$
  - Mark B as executable

  - `visit` all ϕ instructions in B
  - **if** this is the first visit to B**:**
    `visit` all other instructions in B

  - **if** B has a *unique* successor *edge* that is **NOT** executable yet**:**
    Append the *edge* to CFGWorklist

SCCP - How it works

Sparse Conditional Constant Propagation

- SSA Worklist (x is an edge (value) from SSAWorklist)
  - **if** x is a ϕ**:**
    visit(x)
  - **else:**
    if B containing x is executable**:**
    visit(x)

SCCP - How it works

Sparse Conditional Constant Propagation

- visit (Instruction I)
  - **if** I is a ϕ**:**
    Meet all the information from the executable incoming edges
  - **else if** I is a conditional branch**:**
    Evaluate condition (predicate) and append executable CFG edge
  - **else:**
    Update dataflow information (apply transfer function)

  - **if (**old dataflow information of I**) != (**new dataflow information of I**):**
    Append I's all outgoing SSA edges to the SSAWorklist

SCCP Analysis Example

Practice!

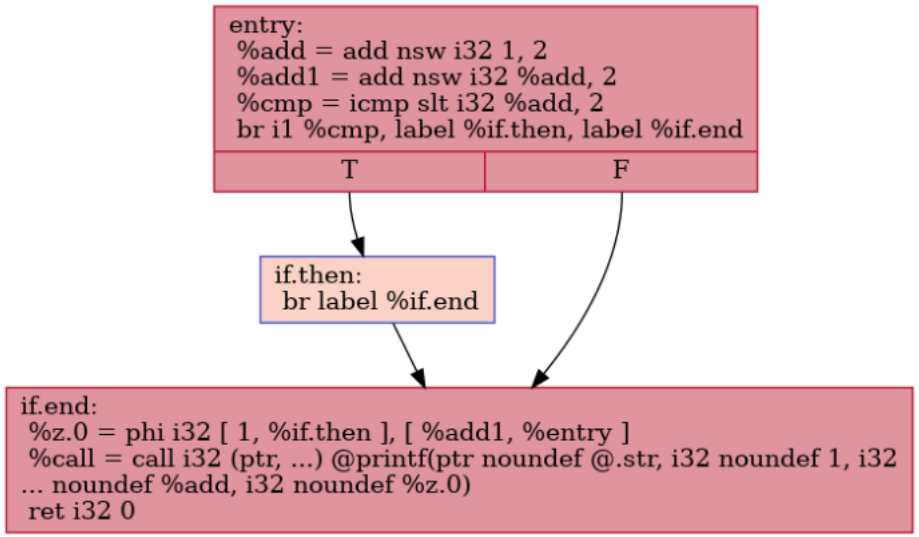
- Compare LLVM IR and source code
- Draw CFG / SSA Graph
- Follow the algorithm and track the values of %add, %add1, %cmp, %z.0

```
; Function Attrs: noinline nounwind uwtable
define dso_local i32 @main() #0 {
entry:
  %add = add nsw i32 1, 2
  %add1 = add nsw i32 %add, 2
  %cmp = icmp slt i32 %add, 2
  br i1 %cmp, label %if.then, label %if.end

if.then:                                          ; preds = %entry
  br label %if.end

if.end:                                           ; preds = %if.then, %entry
  %z.0 = phi i32 [ 1, %if.then ], [ %add1, %entry ]
  %call = call i32 (ptr, ...) @printf(ptr noundef @.str, i32 noundef 1, i32 noundef %add, i32 noundef %z.0)
  ret i32 0
}
```

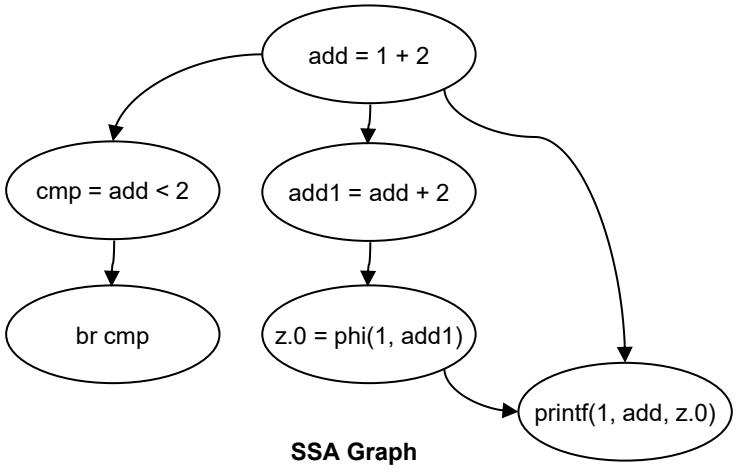
```
#include <stdio.h>

int main() {
  int x = 1;
  int y = x + 2; // 3
  int z = y + 2; // 5

  if (y < 2) { // false
    z = 1;
  }

  // x = 1, y = 3, z = 5
  printf("%d %d %d\n", x, y, z);

  return 0;
}
```

SCCP Analysis Example

1	%z.0 : { 5 }
2	%add : { 3 }
3	%cmp : { 0 }
4	%add1 : { 5 }



CFG for 'main' function



SSA Graph

Algorithm 1 Sparse Conditional Constant Propagation

Input: CFG, SSA graph

```
CFGWorklist ← {Edge(NULL, entry)}
SSAWorklist ← ∅
while (!CFGWorklist.Empty || !SSAWorklist.Empty) do
    x ←pop one from any list
    if x is from CFGWorklist then
        E ← x
        B ←destination(E)
        markExecutable(E)
        visit(φ)    (∀φ ∈ B)
        if B.IsFirstVisit then
            visit(I)    (∀I ∈ B)
        if hasSingleSuccessor(B) then
            S ←getSingleSuccessor(B)
            E′ ←Edge(B, S)
            if !isExecutableEdge(E′) then
                CFGWorklist.append(E′)
    else /* x is from SSAWorklist */
        if x is a φ then
            visit(x)
        else
            if Any incoming CFG edge to x is executable then
                visit(x)
```

Algorithm 2 visit

Input: An instruction I

if I is a ϕ **then**
  Meet all the arguments from the executable edges.
else if I is a conditional `branch` **then**
  Evaluate the condition and append reachable edge to `CFGWorklist`.
else
  Update dataflow information using transfer function.

if Dataflow information of I had been changed **then**
  Append all outgoing SSA edges of I to `SSAWorklist`.

PROJECT 3

Project 3

SimpleSCCP

- 정수 상수와 일부 산술 연산을 분석하는 SCCP pass인 SimpleSCCP를 구현하시오

input

- `mem2reg` pass를 통과하고 난 이후의 LLVM IR

output

- stderr에 분석 결과를 출력

Objective

- 주어진 test.c에 SimpleSCCP를 적용하고, 결과를 예시 출력과 비교한다.
- SCCP가 적용될 수 있는 새로운 testcase.c를 직접 작성한다.
- 보고서 작성
  - 구현체가 pseudocode와 어떻게 대응되는지 코드 리뷰 작성
  - testcase.c에 SimpleSCCP를 적용하고 분석 결과 정리

Project 3

Given Test case

- Conditional constant인 `%z.0 = 5`를 찾아내면 정답

```c
#include <stdio.h>

int main() {
  int x = 1;
  int y = x + 2; // 3
  int z = y + 2; // 5

  if (y < 2) { // false
    z = 1;
  }

  // x = 1, y = 3, z = 5
  printf("%d %d %d\n", x, y, z);

  return 0;
}
```

```
; Function Attrs: noinline nounwind uwtable
define dso_local i32 @main() #0 {
entry:
  %add = add nsw i32 1, 2
  %add1 = add nsw i32 %add, 2
  %cmp = icmp slt i32 %add, 2
  br i1 %cmp, label %if.then, label %if.end

if.then:                                          ; preds = %entry
  br label %if.end

if.end:                                           ; preds = %if.then, %entry
  %z.0 = phi i32 [ 1, %if.then ], [ %add1, %entry ]
  %call = call i32 (ptr, ...) @printf(ptr noundef @.str, i32 noundef 1, i32 noundef %add, i32 noundef %z.0)
  ret i32 0
}
```

Useful Commands

프로젝트 Build

- cmake -S . -G Ninja -B build // Configure cmake
- cmake --build build // Build project

IR 생성

- clang -O0 -S -emit-llvm -Xclang -disable-O0-optnone -Xclang -disable-llvm-passes -fno-discard-value-names test.c -o test.ll

`mem2reg` pass 적용

- opt -S -passes='mem2reg' test.ll -o input.ll

구현한 pass 적용

- opt -S -load-pass-plugin build/lib/libSimpleSCCP.so -passes='print<simple-sccp>' input.ll -disable-output 2> test.out

✓ **직접 test를 위한 source code를 작성하고, SCCP의 결과를 확인**

Appendix

CODE REVIEW

LLVM / C++ Techniques

Curiously Recurring Template Pattern (CRTP)

- 자식 class가 부모 class의 template 인자로 전달되는 패턴
  - **e.g.**,
    - `llvm::AnalysisInfoMixin, llvm::PassInfoMixin,`
  - Virtual table 사용을 줄여 성능 향상을 하기 위함

Mixin

- 여러 기능들을 포함해둔 일종의 base class
  - https://en.wikipedia.org/wiki/Mixin
  - LLVM에서 pass는 `AnalysisInfoMixin` 혹은 `PassInfoMixin`을 사용해 구현

LLVM / C++ Techniques

Substitution Failure Is Not An Error (SIFNAE)

- C++ template의 규칙
- 컴파일 시점 타입, 조건 판단에 주로 사용됨
  - **e.g.**,

```
// Static type checking via SFINAE.
template <typename ValueT, typename = typename std::enable_if<std::is_base_of<
                                      LatticeValue<ValueT>, ValueT>::value>::type>
class SparseDataflowFramework;
```

LLVM Naming rules

- 함수는 lowerCamelCase로
  - **e.g.**, `getOpcodeName, getKey`

- 변수, 타입 이름은 UpperCamelCase로
  - **e.g.**, `PreservedAnalyses, MadeChanges`
  - **c.f.**, MLIR (Google) style - 변수 이름도 lowerCamelCase로

LLVM / C++ Techniques

Pass / Analysis Manager (PM / AM)

- Transform, analysis pass들의 등록, 실행, 결과를 관리하는 object

… InfoMixin::run

- 첫번째 변수의 type (`IRUnitT`)이 적용 범위를 결정
- 두번째 변수로 사용할 수 있는 AM을 받음
  - **e.g.**, `run(llvm::Function &F, llvm::FunctionAnalysisManager &FAM)`

Pass Plugin Registration

- ```
  extern "C" ::llvm::PassPluginLibraryInfo LLVM_ATTRIBUTE_WEAK
      llvmGetPassPluginInfo
  ```

- 정해진 함수에서 pass의 정보가 담긴 구조체를 반환하면 됨
  - 주어진 코드 참고
  ```

SparseDataflowFramework.h

class LatticeValue

- Lattice를 구현하는 class
  - 자식 class (`LVImpl`) 를 CRTP로 넘겨 받을 것을 기대하는 class
  - **c.f.**, `LatticeValue` ← `ConstantValue`

class SparseDataflowFramework

- Dataflow analysis를 위한 base class
  - `LatticeValue`를 상속받은 template 인자 (`ValueT`) 를 기대함

  - `static llvm::AnalysisKey Key`
    - 각 analysis의 고유한 key 값을 만들 때 사용하는 변수
    - 해당 변수의 주소값이 key 값으로 사용됨

getId

- `` `Value`의 이름을 구하는 helper 함수 ``

SimpleSCCP.h

class ConstantValue

- SCCP에서 사용할 lattice를 구현하는 class
  - 정수 값에 대해서만 고려

class SimpleSCCPAnalysis

- 실제로 SCCP의 분석을 수행하는 class

- `DataflowFactsTy`
  - SCCP의 lattice value와 그 값을 가진 value (instruction, block) 쌍

- `struct InstructionVisitor`
  - 각 instruction을 돌면서 `visit`을 수행할 struct
  - LLVM은 여러 종류의 instruction을 순회해야 할 경우, if-else 대신 `InstructionVisitor`를 사용할 것을 권장함

SimpleSCCP.h

class SimpleSCCPAnalysis

- `struct InstructionVisitor`

```
private:
  struct InstructionVisitor
      : public llvm::InstVisitor<InstructionVisitor, ConstantValue> {
    InstructionVisitor(SimpleSCCPAnalysis &Pass) : ThePass(Pass) {}

    ConstantValue visitPHINode(const llvm::PHINode &I);
    ConstantValue visitBranchInst(const llvm::BranchInst &I);
    ConstantValue visitICmpInst(const llvm::ICmpInst &I);
    ConstantValue visitBinaryOperator(const llvm::BinaryOperator &I);

    ConstantValue visitInstruction(const llvm::Instruction &I);

    // An anchor to access the class surrounding the visitor.
    SimpleSCCPAnalysis &ThePass;
  };
```

SimpleSCCP.h

class SimpleSCCPAnalysis

- `run`
  - Module의 각 Function에 대해 실행될 함수
- `analyze` (Algorithm 1)
  - 실제로 SCCP 알고리즘이 실행되는 함수
- `visit` (Algorithm 2)
- `CFGWorkset, SSAWorkset`
  - Worklists

class SimpleSCCPPrinter

- Analysis 결과를 출력하는 pass